h_da
HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

Hochschule Darmstadt

– Fachbereich Informatik –

A Classic Thesis Style

Exposé der Abschlussarbeit zur Erlangung des akademischen Grades
Thema: Evaluation des durch die Instrumentierung mit OpenTelemetry verursachten Overheads in Java- Python- und NodeJS-basierten Serverless-Umgebungen

Bachelor of Science (B. Sc.)

vorgelegt von

Aksel Kenanov

Matrikelnummer: 1118200

Referent	:	Prof. Dr. Peter Kling
Korreferent	:	Prof. Dr. Martin Stiemerling

Aksel Kenanov: *A Classic Thesis Style*, © 8. Dezember 2025

ERKLÄRUNG

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellennachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Darmstadt, 8. Dezember 2025

Aksel Kenanov

INHALTSVERZEICHNIS

I Thesis

1 Problemstellung 2
- 1.1 Einleitung . . . 2
- 1.2 Grundlagen . . . 2
  - 1.2.1 Event Driven Architektur . . . 2
  - 1.2.2 Serverless Architektur . . . 3
  - 1.2.3 Function as a Service (FaaS) . . . 3
  - 1.2.4 Distributed Tracing . . . 4
  - 1.2.5 Observability . . . 4
  - 1.2.6 Opentelemetry . . . 4
  - 1.2.7 Jaeger . . . 6

2 Erkentnisinteresse 7
- 2.1 Einleitung . . . 7
- 2.2 Benchmarks . . . 7
- 2.3 Warum soll das Problem behandelt werden? . . . 8
- 2.4 Originalität der Arbeit . . . 8

3 Fragestellung 9
- 3.1 Leitfrage . . . 9
- 3.2 Teilfragen . . . 9
  - 3.2.1 Wie hoch ist der prozentuale Performance-Overhead in den einzelnen Programmiersprachen? . . . 9
  - 3.2.2 Welche Microbenchmark-Szenarien zeigen die größten sprachspezifischen Unterschiede? . . . 10
  - 3.2.3 Wie skaliert der Overhead mit steigender Anzahl von Funktionsaufrufen? . . . 10
  - 3.2.4 Welche praxisrelevanten Empfehlungen lassen sich ableiten? . . . 10
- 3.3 Abgrenzung und Fokus . . . 10

4 Ziele und Hypothesen 11
- 4.1 Ziele . . . 11
- 4.2 Hypothesen . . . 11

5 Theoriebezug 12
- 5.1 Zentrale theoretische Bezugspunkte . . . 12
- 5.2 Schwerpunkt und Ergänzungen . . . 12
- 5.3 Begründung der Auswahl . . . 12

6 Stand der Forschung 14
- 6.1 Einleitung . . . 14
- 6.2 Relevante Studien . . . 14
- 6.3 Fazit und Forschungslücke . . . 15

7 Methode 16
- 7.1 Methodischer Ansatz . . . 16

7.2 Begründung der Methodenwahl 16
7.2.1 Warum empirisch-quantitativ? 16
7.2.2 Warum experimentelles Design? 17
7.2.3 Warum diese fünf Benchmarks? 17
7.3 Versuchsaufbau und Durchführung 17
7.3.1 Experimentelle Faktoren 17
7.3.2 Messgrößen und Operationalisierung 17
7.3.3 Ablauf der Datenerhebung 18
7.3.4 Statistische Auswertung 18
7.4 Methodische Einschränkungen und Validität 18
7.4.1 Interne Validität 18
7.4.2 Externe Validität 19
7.4.3 Zuverlässigkeit und Reproduzierbarkeit 19
7.5 Kenntnisstand der forschenden Person 19
8 Material 21
8.1 Hardware und Cloud-Infrastruktur 21
8.1.1 Verfügbare Ressourcen 21
8.1.2 Anforderungen und Dimensionierung 21
8.2 Software und Frameworks 21
8.2.1 Verfügbare Softwarekomponenten 21
8.2.2 Entwicklungs- und Auswertungswerkzeuge 22
8.3 Daten und Quellen 22
8.3.1 Verfügbare Primärquellen 22
8.3.2 Verfügbare Sekundärquellen 22
8.4 Noch zu beschaffendes Material 23
8.4.1 Kritische Anforderungen (Must-Have) 23
8.4.2 Wünschenswerte Ergänzungen (Nice-to-Have) 23
8.5 Verfügbarkeit und Kosten 23
8.5.1 Kostenfreie oder im Free Tier enthaltene Ressourcen . . 23
8.5.2 Kostenpflichtige Ressourcen 23
9 Vorläufige Gliederung 24
10 Vorläufiger Projekt- und Zeitplan 25
10.1 Phase 1: Vorbereitung (Tag 1-15) 25
10.2 Phase 2: Implementierung (Tag 16-60) 25
10.3 Phase 3: Experimente (Tag 61-75) 25
10.4 Phase 4: Auswertung & Dokumentation (Tag 76-90) 25

Literatur 26

ABBILDUNGSVERZEICHNIS

Abbildung 2.1 Microbenchmarks [EWL23] 7

TABELLENVERZEICHNIS

Tabelle 7.1 Experimentelle Faktoren und Kontrolle 17

LISTINGS

ABKÜRZUNGSVERZEICHNIS

Teil I

THESIS

1 PROBLEMSTELLUNG

In stark skalierenden, event-getriebenen Serverless-Architekturen stellt Distributed Tracing eine essentielle Observability-Technologie dar. Die vollständige Erfassung aller Traces führt jedoch in manchen Anwendungsfällen zu einem signifikanten Performance-Overhead, der durch die Instrumentierung und das Export der Telemetriedaten entsteht.
Da verschiedene Programmiersprachen aufgrund ihrer Implementierung unterschiedlich stark von diesem Overhead betroffen sind, adressiert diese Arbeit den Vergleich des Performance-Overheads in Java- Python- und NodeJS-basierten Serverless-Anwendungen.
Es ist wichtig diesem Problem nahzugehen, weil OpenTelemetry eines der meistbenutzten Distributed-Tracing-Frameworks ist, der Sprachspezifische Overhead von Opentelemetry in Serverless-Umgebungen bisher noch nicht erforscht ist und weil diese Arbeit somit wichtige Erkentnisse und Empfehlungen für die Praxis ableiten wird, die die Programmiersprachenwahl im Kontext von Distributed Tracing Overhead mit OpenTelemetry deutlich vereinfachen wird.

1.1 EINLEITUNG

In diesem Kapitel werden die grundlegenden Konzepte und Technologien vorgestellt, die für das Verständnis der Problemstellung und der Zielsetzung dieser Arbeit erforderlich sind.

1.2 GRUNDLAGEN

1.2.1 *Event Driven Architektur*

Was ist Event Driven Architektur (Eventgetriebene Architektur)? Es gibt diverse Definitionen für den Begriff Ëvent Driven Architecture", jedoch implizieren alle die gleiche Bedeutung. Laut den Autoren der Amazon Web Services Dokumentation [Ama25b] ist Eventgetriebene Architektur wie folgt erklärt:
„Eine ereignisgesteuerte Architektur nutzt Ereignisse, um entkoppelte Dienste auszulösen und zu kommunizieren, und ist in modernen Anwendungen mit Microservices üblich. Ein Ereignis ist eine Zustandsänderung oder ein Update, zum Beispiel wenn ein Artikel in einen Einkaufswagen auf einer E-Commerce-Website gelegt wird. Ereignisse können entweder den Zustand tragen (der gekaufte Artikel, sein Preis und eine Lieferadresse) oder Ereignisse können Identifikatoren sein (eine Benachrichtigung, dass eine Bestellung versandt wurde).“.

1.2.2 *Serverless Architektur*

1.2.2.1 *Definition*

Laut den Autoren der Google-Cloud-Platform-Dokumentation [Go025] ist eine Serverless Architektur wie folgt definiert:
„Serverlose Architekturen sind darauf ausgelegt, Server und Serververwaltung von den Entwicklungsteams zu abstrahieren. „Serverlos“ bedeutet nicht, dass keine Server vorhanden sind. Stattdessen bezieht sich der Begriff auf die Gesamtentwicklungserfahrung. Aus Sicht eines Entwicklers schreiben Sie einfach den Code und führen ihn aus, ohne sich sonst Sorgen machen zu müssen. Sämtliche Bereitstellung, Hardwarewartung, Software- und Sicherheitsupdates für die Server und andere Serververwaltungsaufgaben verantwortet der Cloud-Anbieter. Darüber hinaus skalieren serverlose Architekturen automatisch entsprechend dem Traffic.“.

1.2.2.2 *Vorteile der Serverless Architektur*

Einige wichtige Vorteile von Serverless Architekturen laut der Google Cloud Platform Dokumentation [Go025]sind vor allem:

- Automatisierung
  Serverlose Lösungen vereinfachen die Verwaltung von Servern durch automatisierte Aufgaben.

- Skalierbarkeit
  Serverlose Lösungen skalieren je nach Traffic automatisch hoch und herunter, ohne dass Sie Feinabstimmungen oder andere manuelle Konfigurationen vornehmen müssen.

- Produktivität
  Mit serverlosem Computing können sich Entwickler auf das Schreiben von Code und das Optimieren der Geschäftslogik konzentrieren, anstatt Zeit für die Serververwaltung aufzuwenden. Entwickler können ihren Code bei Bedarf auch direkt in Ad-hoc-Testumgebungen bereitstellen.

1.2.3 *Function as a Service (FaaS)*

Laut IBM Cloud Docs [IBM25a] ist Function as a Service (FaaS) wie folgt definiert:
„Function as a Service (FaaS) ist ein Cloud-Computing-Service, mit dem Kunden Code als Reaktion auf Ereignisse ausführen können, ohne die komplexe Infrastruktur verwalten zu müssen, die normalerweise mit dem Aufbau und Start von Microservices Anwendungen verbunden ist.

Das Hosten einer Softwareanwendung im Internet erfordert in der Regel die Bereitstellung und Verwaltung eines virtuellen oder physischen Servers

sowie die Verwaltung eines Betriebssystems und von Webserver-Hosting-Prozessen. Im Fall von FaaS wird die Verwaltung der physischen Hardware, des Betriebssystems der virtuellen Maschine und der Webserver-Software automatisch vom Cloud-Service-Provider übernommen. Diese Funktion ermöglicht es Entwicklern, sich ausschließlich auf einzelne Funktionen in ihrem Anwendungscode zu konzentrieren."

1.2.4 *Distributed Tracing*

Amazon Web Services Dokumentation [Ama25c] definiert Distributed Tracing wie folgt:
Distributed Tracing (verteiltes Tracing) ist eine Technologie zur Observability in verteilten Systemen. Beim verteilten Tracing werden Datenanforderungen beobachtet, während sie ein verteiltes System durchlaufen. Eine moderne Microservices-Architektur besteht oft aus mehreren kleinen unabhängigen Komponenten. Diese Komponenten kommunizieren ständig und tauschen Daten mithilfe von APIs aus, um komplexe Aufgaben zu erledigen. Mit verteiltem Tracing können Entwickler den Anforderungspfad über verschiedene Microservices hinweg verfolgen oder visuell verfolgen. Diese Sichtbarkeit hilft bei der Behebung von Fehlern oder bei der Behebung von Bugs und Leistungsproblemen.

1.2.5 *Observability*

Observability oder auf Deutsch Beobachtbarkeit ist: „ ist die Fähigkeit, den internen Zustand eines komplexen Systems allein auf der Grundlage der Kenntnis seiner externen Ausgaben, insbesondere seiner Telemetrie, zu verstehen.

Observability spielt eine entscheidende Rolle bei der Aufrechterhaltung der Verfügbarkeit, Leistung und Sicherheit moderner Softwaresysteme und Cloud-Computing-Umgebungen." - IBM Cloud Docs [IBM25b].

1.2.6 *Opentelemetry*

1.2.6.1 *Definition*

OpenTelemetry ist ein Open-Source-Standard und -Framework zur Sammlung von Telemetriedaten (Logs, Metriken und Traces) aus Cloud-nativen Anwendungen [Ope25a]. Es bietet einheitliche APIs, SDKs und Tools, um die Daten zu instrumentieren, zu erfassen und zu exportieren, wodurch eine herstellerneutrale Überwachung und Analyse verteilter Systeme ermöglicht wird. Ziel ist es, die Leistung und das Verhalten von Systemen besser zu verstehen und Probleme schneller zu identifizieren und zu beheben. Das einzige, das diesem Framework fehlt ist eine Backend-Software die, die exportierten Daten visualisieren kann. Es gibt jedoch einige Backend-Softwares für die Speicherung und Visualisierung von exportierten Daten wie z. B. Jae-

ger, Zipkin, Grafana uvm.

1.2.6.2 *Sampling-Strategien von Opentelemetry-Framework*

Sampling ist der Prozess, bei dem entweder stichprobenartig oder regelbasiert nur eine bestimmte Anzahl an Telemetriedaten instrumentiert, erfasst und exportiert werden [Ope25b]. Dieser Ansatz wird häufig in Software mit sehr hoher Netzlast eingesetzt, um den Overhead zu vermeiden, der durch Instrumentierung, Erfassung und Export verursacht wird. Opentelemetry bietet zwei Sampling-Strategien an:

- Head Sampling
  Head Sampling ist eine Sampling-Technik, bei der die Entscheidung, ob ein Trace aufgezeichnet wird, so früh wie möglich getroffen wird. Die Entscheidung, einen Span oder Trace zu erfassen oder zu verwerfen, erfolgt dabei nicht durch die Betrachtung des gesamten Traces.
  Das gebräuchlichste Verfahren des Head Sampling ist das Consistent Probability Sampling, auch Deterministisches Sampling genannt. In diesem Fall wird die Entscheidung anhand der Trace-ID und des gewünschten Prozentsatzes der zu erfassenden Traces getroffen. Dadurch wird sichergestellt, dass vollständige Traces erfasst werden – ohne fehlende Spans – und zwar mit einer konstanten Rate, beispielsweise 5 Prozent aller Traces. Die Vorteile des Head Sampling sind:
  Einfach zu verstehen, Einfach zu konfigurieren, Effizient, Kann an jedem Punkt der Trace-Sammlungspipeline durchgeführt werden.

- Tail Sampling
  Tail Sampling ist ein Verfahren, bei dem die Entscheidung, ob ein Trace (Ablauf) aufgezeichnet wird, auf Grundlage aller oder der meisten Spans innerhalb des Traces getroffen wird. Tail Sampling bietet die Möglichkeit, Traces anhand spezifischer Kriterien auszuwählen, die aus verschiedenen Teilen eines Traces abgeleitet werden – etwas, das mit Head Sampling nicht möglich ist. Einige Beispiele, wie Tail Sampling eingesetzt werden kann:
  Immer Traces aufzeichnen, die einen Fehler enthalten, Traces basierend auf der gesamten Latenz aufzeichnen, Traces aufzeichnen, die bestimmte Attribute oder Werte in einem oder mehreren Spans enthalten; zum Beispiel mehr Traces von einem neu bereitgestellten Service erfassen, Unterschiedliche Sampling-Raten auf Traces anwenden, abhängig von bestimmten Kriterien, etwa wenn Traces nur von Services mit geringem Datenvolumen stammen im Vergleich zu Traces von Services mit hohem Datenvolumen.

Da das Ziel dieser Arbeit die Evaluation des Instrumentirungs-Overheads in verschidenen Programmiersprachen ist, wird für die Experimente in dieser Arbeit aufgrund seiner Einfachheit das Head Sampling verwendet.

1.2.7 *Jaeger*

Jaeger ist eine Open-Source-Plattform für verteiltes Tracing [Jae25], die zur Überwachung und Fehlerbehebung komplexer Microservices eingesetzt wird. Sie hilft Entwicklern dabei, Leistungsengpässe zu identifizieren, Abhängigkeiten zwischen Services zu analysieren und den Ablauf von Anfragen in einem verteilten System nachzuverfolgen, um die Ursachen von Fehlern und Leistungsproblemen zu finden. Es fungiert als Backend-Software für bereits erfasste und exportierte Telemetriedaten. Es ergänzt somit Opentelemetry.

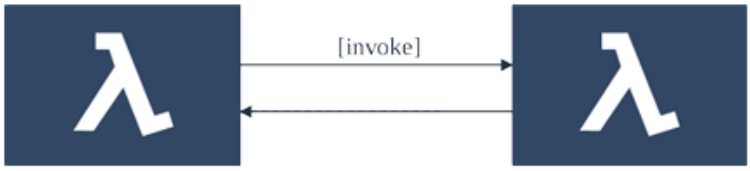
2 ERKENTNISINTERESSE

2.1 EINLEITUNG

Die Hauptintention dieser Arbeit ist es die Messung und Evaluation des prozentualen Anstiegs des Performance-Overheads durch die Instrumentierung mit OpenTelemetry in Serverless-Anwendungen in verschiedenen Programmiersprachen (Java, Python, NodeJS) durchzuführen.

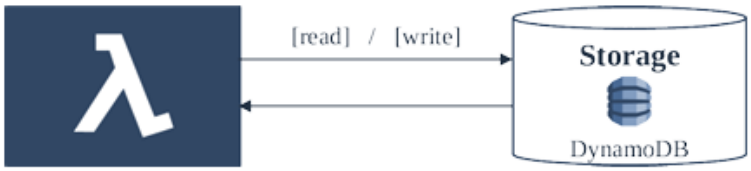
2.2 BENCHMARKS

Im Rahmen dieser Arbeit werden die in der Arbeit [EWL23] vorgestellten Microbenchmarks verwendet und mehrfache Messungen in jeder Programmiersprachen, für jede Benchmark durchgeführt.



(a) Microbenchmark 1: Function Invocation.

(b) Microbenchmark 2 and 3: HTTP GET request (2) and HTTP POST request (3).



(c) Microbenchmarks 4 and 5: DynamoDB read operation (4) and DynamoDB write operation (5).

Abbildung 2.1: Microbenchmarks [EWL23]

2.3 WARUM SOLL DAS PROBLEM BEHANDELT WERDEN?

Wie es bereits in mehreren wissenschaftlichen Arbeiten [Kar24; EWL23; Che+24] beschrieben wurde, verursacht die Instrumentierung mit Telemetrie-Tools einen signifikanten System-Overhead in manchen Anwendungsfällen (für weitere Details siehe Kapitel 6 6).
Die Untersuchung des Performance-Overheads in verschiedenen Programmiersprachen bleibt bisher jedoch unerforscht. Die wissenschaftliche Arbeit der Universitet Bamberg [EWL23] hat den Overhead von verschiedenen Tracing-Tools anhand der obengennten Microbenchmarks gemessen und evaluiert, doch in der Arbeit wurdne nur NodeJS betrachtet, es wurden nur 100 Funktionsaufrufe pro Benchmark betrachtet und im Abschnitt „Conclusion“ wurde erwäht, dass weitere Untersuchungen in versciedenen Sprachen durchgeführt werden sollten um den sprachspezifischen Overhead identifizieren und evaluieren zu können.

2.4 ORIGINALITÄT DER ARBEIT

Die Originalität dieser Arbeit liegt in der Untersuchung des sprachspezifischen Overheads durch die Instrumentierung mit OpenTelemetry in Serverless-Anwendungen für alle 5 obengennte Microbenchmarks.
Des Weiteren werden in dieser Arbeit neue wissenschaftliche Erkenntnisse gewonnen, die, die Entscheidungsfindung für eine Programmiersprache im Kontext von Distributed Tracing erheblich vereinfachen werden.

3 FRAGESTELLUNG

Die vorliegende Arbeit fokussiert sich auf die systematische Untersuchung des Performance-Overheads, der durch die Instrumentierung mit OpenTelemetry in Serverless-Anwendungen entsteht, mit besonderem Augenmerk auf sprachspezifische Unterschiede.

3.1 LEITFRAGE

Wie unterscheidet sich der Performance-Overhead von OpenTelemetry-Instrumentierung in Serverless-Anwendungen zwischen den Programmiersprachen Java, Python und NodeJS, und welche Implikationen ergeben sich daraus für die Praxis?

Diese zentrale Fragestellung adressiert die bisher unerforschte Problematik des sprachspezifischen Overheads in Serverless-Umgebungen. Während bestehende Studien [EWL23] den Overhead einzelner Tracing-Tools untersucht haben, fehlt eine vergleichende Analyse über verschiedene Programmiersprachen hinweg. Die Beantwortung dieser Frage ist essenziell, um fundierte Entscheidungen bei der Technologiewahl für produktive Serverless-Architekturen treffen zu können.

3.2 TEILFRAGEN

Zur detaillierten Beantwortung der Leitfrage werden folgende Teilfragen untersucht:

3.2.1 *Wie hoch ist der prozentuale Performance-Overhead in den einzelnen Programmiersprachen?*

Diese Frage zielt darauf ab, quantitative Messwerte für den Overhead zu erheben, der durch OpenTelemetry-Instrumentierung in Java-, Python- und NodeJS-basierten Lambda-Funktionen entsteht. Dabei werden folgende Metriken betrachtet:

- **Runtime-Overhead:** Prozentuale Erhöhung der Ausführungszeit instrumentierter Funktionen im Vergleich zu nicht-instrumentierten Funktionen
- **Memory-Overhead:** Zusätzlicher Speicherverbrauch während der Funktionsausführung
- **Cold-Start-Overhead:** Auswirkungen auf die Initialisierungszeit bei kalten Starts [Man+18]

3.2.2 *Welche Microbenchmark-Szenarien zeigen die größten sprachspezifischen Unterschiede?*

Die Arbeit nutzt die fünf Microbenchmarks aus der Studie von Eder et al. [EWL23] (siehe Abbildung 2.1), um verschiedene Anwendungsfälle zu simulieren. Diese Teilfrage untersucht:

- In welchen Szenarien (z.B. CPU-intensive vs. I/O-intensive Workloads) treten die größten Unterschiede zwischen den Sprachen auf?
- Gibt es Benchmark-Typen, bei denen eine Sprache besonders gut oder schlecht abschneidet?
- Welche Muster lassen sich bezüglich der Overhead-Verteilung identifizieren?

3.2.3 *Wie skaliert der Overhead mit steigender Anzahl von Funktionsaufrufen?*

Diese Frage adressiert die Skalierungscharakteristik des Overheads:

- Bleibt der prozentuale Overhead konstant oder verändert er sich mit der Anzahl der Aufrufe?

3.2.4 *Welche praxisrelevanten Empfehlungen lassen sich ableiten?*

Die abschließende Teilfrage fokussiert sich auf die praktische Anwendbarkeit der Ergebnisse:

- Für welche Anwendungsfälle ist welche Programmiersprache im Kontext von Distributed Tracing mit OpenTelemetry am besten geeignet?

3.3 ABGRENZUNG UND FOKUS

Die Arbeit konzentriert sich ausschließlich auf:

- **Framework:** OpenTelemetry als Instrumentierungs-Framework
- **Plattform:** AWS Lambda als Serverless-Plattform
- **Sprachen:** Java, Python und NodeJS
- **Metriken:** Performance-Overhead (Runtime, Memory, Cold-Start)

Nicht im Fokus dieser Arbeit sind:

- Vergleich verschiedener Tracing-Frameworks (Zipkin, Jaeger, etc.)
- Funktionale Aspekte der Trace-Qualität
- Kostenanalyse der Cloud-Ressourcen
- Andere Serverless-Plattformen (Azure Functions, Google Cloud Functions)

4 ZIELE UND HYPOTHESEN

In diesem Kapitel werden die Zielsetzung und die überprüfbaren Hypothesen der Arbeit formuliert. Die Ziele sind so gewählt, dass sie innerhalb von 90 Tagen mit einem klaren Experiment- und Auswertungsplan erreichbar sind.

4.1 ZIELE

Die Arbeit verfolgt drei operative Ziele, die direkt auf die Leitfrage (Kapitel 3) einzahlen:

- **Z1 – Messung des Overheads:** Quantitative Bestimmung des Performance-Overheads (Runtime, Memory, Cold-Start) durch OpenTelemetry-Instrumentierung für Java, Python und NodeJS auf AWS Lambda anhand der fünf Microbenchmarks aus [EWL23].
- **Z2 – Vergleich über Benchmarks:** Identifikation der Benchmarks, in denen die größten sprachspezifischen Unterschiede auftreten.
- **Z3 – Praxisempfehlungen:** Ableitung knapper, umsetzbarer Empfehlungen zur Sprach- und Konfigurationswahl für typische Serverless-Szenarien bei gegebenem Overhead-Budget.

4.2 HYPOTHESEN

Die Hypothesen sind so formuliert, dass sie mit den geplanten Messungen (Kapitel 7) innerhalb des Projektzeitraums testbar sind:

- **H1 (Runtime):** Bei den betrachteten I/O-lastigen Benchmarks (Lambda→Lambda, HTTP GET/POST extern, DynamoDB Read/Write) liegt der relative Runtime-Overhead durch OpenTelemetry für Java höher als für Python und für Python höher als für NodeJS.
- **H2 (Memory):** Der zusätzliche Memory-Overhead ist in Java höher als in Python und NodeJS für dieselben Benchmarks, bedingt durch JVM-Overheads.
- **H3 (Cold Start):** OpenTelemetry erhöht die Cold-Start-Latenz in Java stärker als in Python und NodeJS; der Effekt bleibt jedoch unter 25% bei den betrachteten Benchmarks, auch wenn Traces mitsamt Log-Export aufgezeichnet werden.

Die Hypothesen werden als widerlegbar betrachtet und anhand von Mittelwerten und Streuungsmaßen aus mindestens drei wiederholten Testläufen pro Benchmark und Sprache überprüft.

5 THEORIEBEZUG

Dieses Kapitel verankert die Arbeit theoretisch und erläutert, warum die gewählten Konzepte und Modelle für die Untersuchung des sprachspezifischen Overheads von OpenTelemetry in Serverless-Umgebungen geeignet sind.

5.1 ZENTRALE THEORETISCHE BEZUGSPUNKTE

- **OpenTelemetry-Architektur und Sampling-Modell**[Ope25a; Ope25b]: definiert Head-/Tail-Sampling, Export-Pipeline und die Rollen von SDK und Collector; bildet die Grundlage für die Instrumentierung.
- **Messmethodik**[EWL23]: bietet ein reproduzierbares Benchmark-Set (Lambda->Lambda, HTTP GET/POST extern, DynamoDB Read/Write) zur Abbildung typischer I/O-lastiger Serverless-Szenarien.
- **Performance-Overhead**[Kar24]: beschreiben, wie Tracing-Instrumentierung CPU-, Memory- und Netzwerk-Ressourcen beansprucht und die Laufzeit beeinflusst.
- **AWS Distro for OpenTelemetry (ADOT)**[Ama25a]: stellt die konkrete Implementierung von OpenTelemetry in AWS Lambda dar und definiert die Konfigurationsmöglichkeiten für Sampling-Strategien.
- **Lambda Metriken**[Ama25d]: erläutert das Monitoring von Lambda-Funktionen mittels CloudWatch-Metriken (Duration, Memory Usage, Init Duration) zur Erfassung des Overheads.

5.2 SCHWERPUNKT UND ERGÄNZUNGEN

Im Mittelpunkt steht die OpenTelemetry-Architektur und deren Sampling-Modell als primäres Messobjekt. Die Arbeit von Eder et al. liefert das reproduzierbare Benchmark-Framework für die fünf I/O-lastigen Szenarien. Die Performance-Overhead-Modelle von Karkan und Eder at al. dienen als Referenz für Metrik-Definitionen (Runtime, Memory, Cold Start). AWS ADOT und CloudWatch Lambda-Metriken bilden die technische Grundlage für die konkrete Implementierung und Messung in der AWS-Lambda-Umgebung.

5.3 BEGRÜNDUNG DER AUSWAHL

- **Praxisrelevanz und Übertragbarkeit:** OpenTelemetry ist der de-facto-Standard für Cloud-native Observability; AWS ADOT ist die offizi-

elle Lambda-Integration. Die Benchmarks bilden typische Serverless-Szenarien ab (Lambda-Verkettung, HTTP-Calls, DB-Zugriffe).

- **Messbarkeit:** Alle theoretischen Konstrukte sind direkt operationalisierbar über CloudWatch-Metriken (Duration, Memory, Init Duration) und können statistisch ausgewertet werden.
- **Sprachspezifischer Fokus:** Die gewählten Modelle ermöglichen den systematischen Vergleich zwischen Java, Python und NodeJS, da sie runtime-neutral definiert sind und auf alle drei Sprachen anwendbar bleiben.

6 STAND DER FORSCHUNG

In diesem Kapitel wird der aktuelle Stand der Forschung zu dem in dieser Arbeit behandelten Thema dargestellt. Es werden relevante Studien vorgestellt, die als Grundlage für die weitere Untersuchung dienen.

6.1 EINLEITUNG

In diesem Kapitel werden die wichtigsten Beiträge zum Thema zusammengefasst.

6.2 RELEVANTE STUDIEN

- **Evaluation of Programming Languages for Memory Usage, Scalability, and Cold Start, on AWS Lambda Serverless Platform as a Case Study** [Rod+24]:
  In dieser Studie wird anhand einer standarrisierten Architektur die Performance von verschiedenen Programmiersprachen (Java, Python, NodeJS) auf AWS Lambda für CRUD-Operationen untersucht. Dabei werden Metriken wie Skalierbarkeit, Cold-Start-Zeiten, Runtime für Leseoperationen analysiert.
  Java zegit die längsten Cold-Start-Zeiten für alle CRUD-Operationen, jedoch sind die Metriken Runtime und Skalierbarkeit bei Hot-Starts sehr variabel.
  Je nach Anzahl der gleichzeitigen Ausführungen (Concurrent Executions also Lambda Concurrency) ist die Execution Time für Leseoperationen in Java oder In Python am besten, wobei NodeJS tendenziell immmer entweder die zweitbeste oder die schlechteste Performance zeigt.
  Java zeigt die beste Performance bei hohem verfügbaren Speicher und eine erheblich schlechtere Performance bei niedrigem Speicher. insgesamt ist die Performance von den drei Sprachen jedoch sehr nuanciert und hängt stark von der Anzahl der gleichzeitigen Ausführungen und dem verfügbaren Speicher ab.
  Diese Arbeit liefert leider wiedersprüchliche Ergebnisse zur Runtime für Leseoperationen da sie an zwei verschidenen Stellen unterschiedliche Ergebnisse präsentiert. Daher ist es auch nicht klar nachvollzierbar, ob Java oder Python die bessere Performance bei Leseoperationen zeigt. Das einzige klare Ergebnis ist, dass Java die längsten Cold-Start-Zeiten hat.

- **A Comparison of Distributed Tracing Tools in Serverless Applications** [EWL23]:
  In dieser Arbeit wird der Performance-Overhead von verschiedenen Distributed Tracing Tools (OpenTelemetry, Zipkin usw.) für die 5 Benchmarks aus Abbildung 2.1 in NodeJS auf AWS Lambda untersucht. Meine Arbeit baut auf dieser Studie auf, indem sie den Overhead von OpenTelemetry in verschiedenen Programmiersprachen (Java, Python, NodeJS) anhand der gleichen Benchmarks vergleicht. Die Zentrale Fragestellung meiner Arbeit stammt aus dem Abschnitt "Conclusion"dieser Studie, in dem die Autoren vorschlagen, den sprachspezifischen Overhead von Tracing-Tools zu untersuchen.

Desweiteren ist der Fakt, dass Instrumentierung mit Telemetrie-Tools einen Overhead verursacht in mehreren wissenschaftlichen Arbeiten beschrieben worden [Kar24; Che+24; EWL23; NN25; Zai25].

6.3 FAZIT UND FORSCHUNGSLÜCKE

Die bisherige Forschung hat gezeigt, dass die Instrumentierung mit Telemetrie-Tools wie OpenTelemetry einen Performance-Overhead in Serverless-Anwendungen verursachen kann. Allerdings wurde der sprachspezifische Overhead bisher noch nicht untersucht. Meine Arbeit zielt darauf ab, diese Forschungslücke zu für die Programmiersprachen Java, Python und NodeJS zu schließen.

7 METHODE

Dieses Kapitel beschreibt die wissenschaftliche Vorgehensweise zur Beantwortung der Forschungsfragen (Kapitel 3) und zur Überprüfung der Hypothesen (Kapitel 4). Die Methodik ist so gewählt, dass sie innerhalb von 90 Tagen durchführbar ist und reproduzierbare, statistisch auswertbare Ergebnisse liefert.

7.1 METHODISCHER ANSATZ

Die Arbeit verwendet eine **empirisch-quantitative Methode** mit experimentellem Design:

- **Empirisch:** Primärdatenerhebung durch kontrollierte Messungen in einer realen Cloud-Umgebung (AWS Lambda)
- **Quantitativ:** Erhebung numerischer Performance-Metriken (Runtime, Memory, Cold-Start-Zeit) zur statistischen Auswertung
- **Experimentell:** Systematische Variation der unabhängigen Variablen (Programmiersprache, Benchmark-Typ, Instrumentierungsstatus) unter kontrollierten Bedingungen
- **Vergleichend:** Cross-Language-Analyse über drei Programmiersprachen (Java, Python, NodeJS) und fünf Benchmark-Szenarien

7.2 BEGRÜNDUNG DER METHODENWAHL

7.2.1 *Warum empirisch-quantitativ?*

Die Leitfrage nach sprachspezifischen Performance-Unterschieden erfordert **messbare, vergleichbare Daten**. Theoretische Analysen könnten keine präzisen Aussagen über prozentuale Overhead-Werte liefern. Die empirische Messung ermöglicht:

- Direkte Quantifizierung des Overheads in Prozent (für H1, H2, H3)
- Objektiven Vergleich zwischen Sprachen ohne Bias
- Reproduzierbarkeit und Nachprüfbarkeit durch andere Forschende
- Statistische Analyse zur Validierung der Hypothesen

7.2.2 *Warum experimentelles Design?*

Das Experiment erlaubt die **Isolierung des OpenTelemetry-Effekts** durch Kontrollgruppen (nicht-instrumentierte Lambda-Funktionen). Dies ist notwendig, um kausale Aussagen über den durch Tracing verursachten Overhead zu treffen reine Beobachtungsstudien in Produktionsumgebungen könnten nicht zwischen Tracing-Overhead und anderen Faktoren unterscheiden.

7.2.3 *Warum diese fünf Benchmarks?*

Die Microbenchmarks aus Eder et al. [EWL23] wurden gewählt, weil sie:

- **Repräsentativ** für typische Serverless-Workloads sind (Lambda-Verkettung, HTTP-API-Calls, DB-Zugriffe)
- **I/O-lastig** sind und damit realistische Serverless-Szenarien abbilden (Lambda ist selten CPU-bound)
- **Bereits validiert** sind in einer wissenschaftlichen Studie und sich als sensitiv für Overhead-Messungen erwiesen haben
- **Reproduzierbar** implementiert werden können in allen drei Zielsprachen

7.3 VERSUCHSAUFBAU UND DURCHFÜHRUNG

7.3.1 *Experimentelle Faktoren*

Faktor	Ausprägungen	Kontrolle
Programmiersprache	Java, Python, NodeJS	Variiert
Benchmark-Typ	5 Microbenchmarks	Variiert
Instrumentierung	Mit/Ohne OpenTelemetry	Variiert
Lambda-Memory	1024 MB	Konstant
AWS-Region	eu-central-1	Konstant
Wiederholungen	3 Testläufe	Konstant

Tabelle 7.1: Experimentelle Faktoren und Kontrolle

Wenn die Dozenten der Meinung sind, dass die Messungen zu viel Zeit in Anspruch nehmen, kann die Anzahl der Wiederholungen auf 2 reduziert werden, oder es kann jeweils nur 1 Messung pro Benchmark und Sprache durchgeführt werden.

7.3.2 *Messgrößen und Operationalisierung*

Die Performance-Metriken werden über AWS CloudWatch erfasst [Ama25d]:

- **Runtime (Duration):** Gesamtausführungszeit einer Lambda-Invokation in ms
- **Memory (Max Memory Used):** Höchster Speicherverbrauch während der Ausführung in MB
- **Cold-Start-Latenz (Init Duration):** Zeit für die Initialisierung bei kaltem Start in ms

Der **relative Overhead** wird berechnet als:

$$\text{Overhead}_{\text{Metrik}} = \frac{\text{Metrik}_{\text{mit OTel}} - \text{Metrik}_{\text{ohne OTel}}}{\text{Metrik}_{\text{ohne OTel}}} \times 100\%$$

7.3.3 *Ablauf der Datenerhebung*

1. **Implementierung:** Entwicklung der fünf Benchmarks in Java, Python und NodeJS (insgesamt 30 Lambda-Funktionen mit und ohne Instrumentierung)
2. **Baseline-Messung:** Ausführung ohne OpenTelemetry-Instrumentierung (3 Wiederholungen pro Benchmark/Sprache = 45 Messungen)
3. **Instrumentierung:** Integration von AWS Distro for OpenTelemetry (ADOT) [Ama25a] mit Head-Sampling (100% Sample-Rate für maximalen Overhead)
4. **Instrumented-Messung:** Ausführung mit OpenTelemetry (3 Wiederholungen = 45 Messungen)
5. **Cold-Start-Messung:** Separate Betrachtung der Initialisierung auf AWS CloudWatch

Jede Messung umfasst mindestens 100 Lambda-Invokations, um statistische Stabilität zu gewährleisten.

7.3.4 *Statistische Auswertung*

Die erhobenen Daten werden wie folgt analysiert:

- **Deskriptive Statistik:** Mittelwert, Median, Standardabweichung pro Sprache/Benchmark/Metrik
- **Vergleichende Analyse:** Overhead-Differenzen zwischen den Sprachen visualisiert durch Box-Plots und Bar-Charts

7.4 METHODISCHE EINSCHRÄNKUNGEN UND VALIDITÄT

7.4.1 *Interne Validität*

Kontrolle von Störvariablen:

- Lambda-Memory konstant für alle Setups - verhindert Memory-bedingte Laufzeitunterschiede
- Gleiche AWS-Region - minimiert Netzwerklatenzvarianz
- Wiederholte Messungen → reduziert zufällige Schwankungen

Limitation: Cold-Start-Frequenz in AWS Lambda ist nicht vollständig kontrollierbar; Cold Starts werden daher gezielt durch ausreichend lange Pausen zwischen Invokations erzwungen.

7.4.2 *Externe Validität*

Generalisierbarkeit:

- Ergebnisse gelten für AWS Lambda mit ADOT; Übertragbarkeit auf andere Serverless-Plattformen (Azure Functions, Google Cloud Functions) ist eingeschränkt
- Die fünf Benchmarks decken typische I/O-Szenarien ab, aber nicht alle möglichen Serverless-Workloads (z.B. ML-Inferenz, Video-Verarbeitung)
- Head-Sampling mit 100% stellt den maximalen Overhead dar; niedrigere Sample-Raten würden geringeren Overhead zeigen

7.4.3 *Zuverlässigkeit und Reproduzierbarkeit*

Durch die Dokumentation der exakten OpenTelemetry-Versionen, Lambda-Runtimes und Benchmark-Implementierungen sowie die Verwendung von Infrastructure-as-Code (AWS SAM/CloudFormation) ist das Experiment reproduzierbar.

7.5 KENNTNISSTAND DER FORSCHENDEN PERSON

Die Durchführung der Methode erfordert folgende Kenntnisse, über die der Autor (teilweise) verfügt:

- **AWS Lambda:** Entwicklung und Deployment von Serverless-Funktionen, Konfiguration von Runtimes und Memory-Einstellungen
- **OpenTelemetry:** Instrumentierung von Anwendungen, Konfiguration von Sampling-Strategien, Integration mit AWS ADOT
- **Programmierung:** Implementierung in Java, Python und NodeJS mit entsprechenden AWS SDKs
- **CloudWatch:** Erfassung und Export von Performance-Metriken, Erstellung von Dashboards
- **Statistische Auswertung:** Statistik & Stochastik.

Die Kenntnisse wurden durch Studienmodule sowie durch praktische Erfahrung in AWS-Projekten erworben.

8 MATERIAL

Dieses Kapitel beschreibt das zur Durchführung der Arbeit erforderliche Material, die eingesetzten Technologien und verfügbaren Datenquellen.

8.1 HARDWARE UND CLOUD-INFRASTRUKTUR

8.1.1 *Verfügbare Ressourcen*

- **Entwicklungs-PC:** Lokale Umgebung für Entwicklung, Testing und Scripting (Windows/Linux/Mac möglich)
- **AWS Account:** AWS Free Tier mit verfügbarem Budget von 200€ für Cloud-Ressourcen und Datenübertragung
- **AWS Lambda:** Serverless Compute für die Ausführung der Benchmarks in allen drei Runtimes (Java, Python, Node.js)
- **Amazon CloudWatch:** Monitoring und Log-Erfassung für Performance-Metriken (Duration, Memory Usage, Init Duration)
- **Amazon DynamoDB (Free Tier):** Für die DB-Read/Write-Benchmarks im Speicher-Szenario

8.1.2 *Anforderungen und Dimensionierung*

- Lambda-Speicher: 1024 MB konstant pro Funktion (ausreichend für alle Benchmarks)
- Geschätzter Kostenbedarf: 200€ deckt ca. 50–100 Millionen Lambda-Ausführungen (mehr als ausreichend für 15 Funktionen × 100 Invokationen × 3 Wiederholungen × 2 Szenarien = 90.000 Invokationen)
- Speicher für Ergebnisse: Lokal oder über AWS S3 (ebenfalls im Free Tier)

8.2 SOFTWARE UND FRAMEWORKS

8.2.1 *Verfügbare Softwarekomponenten*

- **AWS Distro for OpenTelemetry (ADOT) [Ama25a]:** Offizielle OpenTelemetry-Integration für AWS Lambda, vorkonfiguriert für CloudWatch-Export
- **OpenTelemetry SDKs [Ope25a]:**

  - OpenTelemetry Java SDK (maven/gradle)
  - OpenTelemetry Python SDK (pip)
  - OpenTelemetry JavaScript/Node.js SDK (npm)

- **AWS Lambda Runtimes:** Java 17, Python 3.11, Node.js 20 (aktuelle, unterstützte Versionen)
- **AWS SAM (Serverless Application Model):** Infrastruktur-as-Code für Lambda-Deployment

8.2.2 *Entwicklungs- und Auswertungswerkzeuge*

- **Programmiersprachen:** Java (JDK 17), Python (3.11), Node.js (v20)
- **Paketmanager:** Maven/Gradle (Java), pip (Python), npm (Node.js)
- **Datenauswertung:** Python (pandas, scipy, matplotlib) für statistische Analyse und Visualisierung
- **Versionskontrolle:** Git für Benchmark-Code und Dokumentation

8.3 DATEN UND QUELLEN

8.3.1 *Verfügbare Primärquellen*

- **CloudWatch Metriken:** Rohmetriken aus AWS Lambda-Invokationen (Duration, Memory, Init Duration) direkt verfügbar nach Experiment-Durchführung
- **ADOT Dokumentation [Ama25a]:** Konfigurations- und Best-Practice-Beispiele für Lambda-Instrumentation

8.3.2 *Verfügbare Sekundärquellen*

- **OpenTelemetry Dokumentation [Ope25a]:** Umfassende Referenzen für Sampling-Konfiguration und SDK-Nutzung
- **Jaeger/Jaeger-Backend [Jae25]:** Dokumentation für Trace-Speicherung und optionale erweiterte Analysen
- **AWS Lambda Best Practices [Ama25a]:** Cold-Start-Optimierung, Memory-Sizing, Tracing-Setup
- **Bisherige Arbeiten:** Karkan (2024) [Kar24], Eder et al. (2023) [EWL23] — verfügbar über Universiätsbibliothek oder Preprints

8.4 NOCH ZU BESCHAFFENDES MATERIAL

8.4.1 *Kritische Anforderungen (Must-Have)*

- **AWS Account mit Free Tier:** Bereits vorhanden Zugriff auf Lambda, CloudWatch, DynamoDB
- **Lokale Entwicklungsumgebung:** Java 17 SDK, Python 3.11, Node.js v20 — bereits vorhanden
- **AWS CLI und AWS SAM CLI:** Zur Automation von Lambda-Deployment; Installation und Konfiguration erforderlich
- **Benchmark-Code:** Implementierung oder Anpassung der fünf Szenarien in Java, Python, Node.js — muss noch implementiert werden.

8.4.2 *Wünschenswerte Ergänzungen (Nice-to-Have)*

- **k6 Testskripte:** Für reproduzierbare Lastprofile (alternative: manuelle Invokation via AWS CLI oder boto3)
- **Jaeger UI:** Lokale Jaeger-Instanz zur erweiterten Trace-Analyse (optional, CloudWatch-Daten sind ausreichend)
- **Monitoring-Dashboard:** AWS CloudWatch Dashboard mit vordefinierten Graphen für Visualisierung während Experiment-Durchführung

8.5 VERFÜGBARKEIT UND KOSTEN

8.5.1 *Kostenfreie oder im Free Tier enthaltene Ressourcen*

- OpenTelemetry SDKs, AWS ADOT: Open-Source, kostenfrei
- AWS Lambda (Free Tier): 1 Mio. kostenlose Anfragen/Monat
- CloudWatch (Free Tier): 10 GB Logs kostenlos pro Monat
- Python, Java, Node.js Runtimes: Kostenfrei
- Git, VS Code, Python-Libraries (pandas, scipy): Open-Source/kostenfrei

8.5.2 *Kostenpflichtige Ressourcen*

- AWS Lambda (über Free Tier hinaus): 0,0000002 USD pro Anfrage
- CloudWatch Logs (über Free Tier hinaus): 0,50 USD pro GB
- Datenübertragung aus AWS: 0,09 USD pro GB
- **Gesamtbudget 200€**: Ausreichend für alle Messungen und Export von Ergebnissen

9 VORLÄUFIGE GLIEDERUNG

In diesem Kapitel wird die vorläufige Gliederung der Arbeit dargestellt, um einen strukturierten Überblick über die geplanten Inhalte und deren Anordnung zu geben.

1. Einleitung und Motivation
2. Grundlagen von Distributed Tracing und Serverless-Architekturen
3. Stand der Forschung zu Performance-Overhead in Serverless-Anwendungen
4. Methodik: Experiment-Design und Implementierung
5. Ergebnisse: Performance-Metriken
6. Diskussion: Analyse der Ergebnisse
7. Implikationen für die Praxis
8. Fazit und Ausblick

10 VORLÄUFIGER PROJEKT- UND ZEITPLAN

Das Projekt ist in vier Hauptphasen unterteilt, die jeweils spezifische Aufgaben und Meilensteine umfassen. Während allen Phasen wird der entsprechende Text zur Aufgabe verfasst.Der vorläufige Zeitplan für die Durchführung des Projekts ist wie folgt:

10.1 PHASE 1: VORBEREITUNG (TAG 1-15)

- Vertiefende Literaturrecherche
- Verfassen der ersten 3 Kapitel der Arbeit (Einleitung, Theorethische Grundlagen, Stand der Forschung)

10.2 PHASE 2: IMPLEMENTIERUNG (TAG 16-60)

- Implementierung der Benchmark-Funktionen in Java, Python und NodeJS
- Integration von OpenTelemetry in alle Funktionen
- Verfassen des Methodik-Kapitels

10.3 PHASE 3: EXPERIMENTE (TAG 61-75)

- Durchführung der Experimente gemäß Versuchsaufbau
- Datensammlung und -aufbereitung
- Verfassen des Ergebniskapitels

10.4 PHASE 4: AUSWERTUNG & DOKUMENTATION (TAG 76-90)

- Ausführliche Datenanalyse
- Diskussion der Ergebnisse
- Verfassen der Kapitel Diskussion, Implikationen und Fazit

LITERATUR

[Ama25a] Amazon Web Services. *AWS Lambda Best Practices, AWS Distro for OpenTelemetry – Distributed Tracing*. Accessed: 2025-12-07. 2025. URL: https://docs.aws.amazon.com/lambda/latest/dg/best-practices.html.

[Ama25b] Amazon Web Services. *Event-Driven Architectures on AWS*. Accessed: 2025-12-07. 2025. URL: https://aws.amazon.com/de/event-driven-architecture/.

[Ama25c] Amazon Web Services. *Implementing Distributed Tracing – AWS Distro for OpenTelemetry*. Accessed: 2025-12-07. 2025. URL: https://docs.aws.amazon.com/otel/latest/adot/lambda-tracing.html.

[Ama25d] Amazon Web Services. *Monitor AWS Lambda with Amazon CloudWatch*. Accessed: 2025-12-07. 2025. URL: https://docs.aws.amazon.com/lambda/latest/dg/monitoring-metrics.html.

[Che+24] Zhuangbin Chen, Zhihan Jiang, Yuxin Su, Michael R. Lyu und Zibin Zheng. "Tracemesh: Scalable and Streaming Sampling for Distributed Traces". In: *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*. 2024, S. 54–65. DOI: 10.1109/CLOUD62652.2024.00016.

[EWL23] Christina Eder, Stefan Winzinger und Robin Lichtenthäler. "A Comparison of Distributed Tracing Tools in Serverless Applications". In: *2023 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. 2023, S. 98–105. DOI: 10.1109/SOSE58276.2023.00018.

[Goo25] Google Cloud. *Serverless Architectures*. Accessed: 2025-12-07. 2025. URL: https://cloud.google.com/discover/what-is-serverless-architecture.

[IBM25a] IBM Cloud. *What is Function as a Service (FaaS)?* Accessed: 2025-12-07. 2025. URL: https://www.ibm.com/de-de/think/topics/faas.

[IBM25b] IBM Cloud. *What is Observability?* Accessed: 2025-12-07. 2025. URL: https://www.ibm.com/de-de/think/topics/observability.

[Jae25] Jaeger Project. *Jaeger Documentation*. Accessed: 2025-12-07. 2025. URL: https://www.jaegertracing.io/docs/.

[Kar24] T. M. Karkan. "Performance Overhead Of OpenTelemetry Sampling Methods In A Cloud Infrastructure". In: (2024). Published 2024.

[Man+18] Johannes Manner, Martin Endreß, Tobias Heckel und Guido Wirtz. "Cold Start Influencing Factors in Function as a Service". In: *2018 IEEE/ACM International Conference on Utility and Cloud Computing Companion (UCC Companion)*. 2018, S. 181–188. DOI: 10.1109/UCC-Companion.2018.00054.

[NN25] Vorname Nachname und Vorname2 Nachname2. *Optimizing Distributed Tracing: Best practices for remaining within budget and capturing critical traces*. Datadog Architecture Center. Accessed: 2025-12-07. 2025. URL: https://www.datadoghq.com/architecture/optimizing-distributed-tracing-best-practices-for-remaining-within-budget-and-capturing-critical-traces/.

[Ope25a] OpenTelemetry Community. *OpenTelemetry Documentation*. Accessed: 2025-12-07. 2025. URL: https://opentelemetry.io/docs/.

[Ope25b] OpenTelemetry Community. *OpenTelemetry Official Documentation: Sampling Configuration*. Accessed: 2025-12-07. 2025. URL: https://opentelemetry.io/docs/concepts/sampling/.

[Rod+24] Nelson Rodríguez, María Murazzo, Adriana Martín und Matías Rodríguez. "Evaluation of Programming Languages for Memory Usage, Scalability, and Cold Start, on AWS Lambda Serverless Platform as a Case Study". In: *Computer Science – CACIC 2023*. Hrsg. von Patricia Pesado, Walter Panessi und Juan Manuel Fernández. Bd. 2123. Communications in Computer and Information Science. First online: 23 June 2024. Springer, Cham, 2024, S. 33–45. DOI: 10.1007/978-3-031-62245-8_3.

[Zai25] Mohammad Zaigam. *Distributed Tracing Guide: Best Practices 2025*. Accessed: 2025-12-07. 2025. URL: https://www.apica.io/blog/what-is-distributed-tracing-how-it-works-and-best-practices.